rized the early efforts in this domain, noting that, with the gradual adoption of commercial FPGA devices, pioneering research began to explore their potential for SAT solving. Notable early works include Suyama et al. [9], Plazner [5], and Zhong [4], among others. These works introduced the first category of FPGA-based SAT solvers, referred to as *instance-specific* implementation. The limited hardware resources and the need for frequent reconfiguration led to the decline of this approach.

### B. Recent Works

Several notable advancements in FPGA-based SAT solving emerged after 2017, as summarized by Sohanghpurwala. [23]. During this period, research focused on both incomplete and complete SAT solving algorithms, exploiting FPGA's parallelism capabilities. Early FPGA implementations, including those based on local search algorithms like WSAT, demonstrated the potential for hardware acceleration in SAT solving.

For incomplete SAT algorithms, FPGA implementations of *WSAT* were explored in [12] and [15], while [20] describes an FPGA implementation of *ProbSAT*, another local search-based approach. These works highlighted FPGA's ability to efficiently explore the solution space; however, the limitations of local search algorithms meant that these solutions could not guarantee satisfiability or unsatisfiability.

As FPGA hardware matured, many FPGA-based complete SAT solvers adopted heterogeneous architectures, using FPGAs as accelerators for specific parts of the CDCL algorithm. Operations such as unit propagation and clause learning were recognized for their potential to be parallelized or pipelined, which led to several efforts in FPGA-based acceleration. For example, [21] and [14] attempted to accelerate the unit propagation stage by creating multiple hardware threads, while [16] used FPGAs to speed up the conflict analysis procedure.

For complete SAT solvers implemented directly in hardware, a batch of DPLL-based designs are proposed , such as in [31] and [19]. These implementations focus on embedding the DPLL algorithm into FPGA hardware for more efficient SAT solving. A recent contribution in this direction is SAT-Hard [26], which proposes a monolithic state-machine-based implementation of the full CDCL framework.

SAT solving.

## IV. VeriSAT

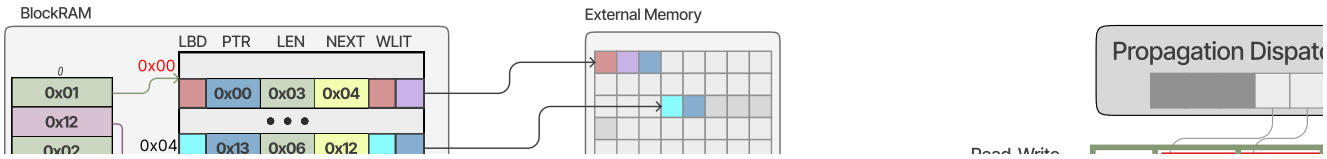
### A. Overview of VeriSAT design

As depicted in Figure 1, The solving l
be organized as three principal modules, e
step of the CDCL framework:

- **Propagation Search Engine (PSE)**:
straint Propagation (BCP). PSE scan
conflicts or unit clauses. Upon encou
the Conflict Analysis Engine; if no
clauses remain, it hands control to th
- **Conflict Analysis Engine (CAE)**:
when PSE reports a conflict. CAE
graph to derive a learned clause and
level. If the backtracking level is ne
UNSAT. Otherwise, CAE appends
clause database, backjumps to the co
returns the control to PSE.
- **Variable Decision Engine (VDE)**:
decisions. VDE applies a dynamic
to pick an unassigned variable and
all variables are assigned without co
SAT. Otherwise, VDE updates the h
invokes PSE to continue propagation

It should be noted that, based on the
CDCL algorithm, all three main modul
maintain the correctness and completene
other hardware pipelining schemes that r
run in the same time window, making any
work concurrently will violate the correc
the algorithm.

### B. Memory Scheme in VeriSAT

The memory scheme of **VeriSAT** is de
and memory storage as much as possible,
ule owning its respective memory. Unlike

Authorized licensed use limited to: Duke University. Downloaded on March 25,2026 at 21:03:02 UTC from IEEE Xplore.